

Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
saikrishna@DESKTOP-LGFVDTJ:~$ nano task1.c
saikrishna@DESKTOP-LGFVDTJ:~$ nano skill1.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc skill1.c -o skill
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill.c
-bash: ./skill.c: No such file or directory
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill
myShell> ls
Parsing Result:
Argument 1: ls
saikrishna@DESKTOP-LGFVDTJ:~$ echooo
echooo: command not found
saikrishna@DESKTOP-LGFVDTJ:~$ cat skill1.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];
    char *token;

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    token = strtok(input, " \n");

    if (token == NULL) {
        printf("Empty command\n");
        return 0;
    }

    printf("Parsing Result:\n");

    int i = 1;

    while (token != NULL) {
        printf("Argument %d: %s\n", i, token);
        token = strtok(NULL, " \n");
        i++;
    }

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ |
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
saikrishna@DESKTOP-LG X Windows PowerShell X
saikrishna@DESKTOP-LGFVDTJ:~$ nano skill2.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc skill2.c -o skill2
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill2
myShell> ls
Parsing Result:
ls
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill2
myShell> hello world
Parsing Result:
hello
world
saikrishna@DESKTOP-LGFVDTJ:~$ cat skill2.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    int count = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\')
            count++;
    }

    if (count % 2 != 0) {
        printf("Syntax Error: Unmatched single quote.\n");
        return 0;
    }

    printf("Parsing Result:\n");

    char *token = strtok(input, " \\n");

    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, " \\n");
    }

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ |
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ nano skill3.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc skill3.c -o skill3
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill3
myShell> ls
Parsing Result:
Argument 1: echo
Argument 2: Hello saikrishna
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill3
myShell> echo "Hello $USER"
Parsing Result:
Argument 1: echo
Argument 2: Hello saikrishna
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill3
myShell> hello hostname
Parsing Result:
Argument 1: echo
Argument 2: Hello saikrishna
saikrishna@DESKTOP-LGFVDTJ:~$ cat skill3.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char input[200];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    int quote = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '"')
            quote++;
    }

    if (quote % 2 != 0) {
        printf("Syntax Error: Unmatched double quote.\n");
        return 0;
    }

    printf("Parsing Result:\n");

    printf("Argument 1: echo\n");
    printf("Argument 2: Hello %s\n", getenv("USER"));

    return 0;
}

```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```

}
saikrishna@DESKTOP-LGFVDTJ:~$ nano skill4.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc skill4.c -o skill4
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill4
myShell> ls
Parsed Output: ls
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill4
myShell> hello user
Parsed Output: hello user
saikrishna@DESKTOP-LGFVDTJ:~$ cat skill4.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char output[200];

    printf("myShell> ");
    fgets(input, sizeof(input), stdin);

    int i = 0, j = 0;

    while (input[i] != '\0') {
        if (input[i] == '\\' && input[i + 1] != '\0') {
            i++;
            output[j++] = input[i];
        }
        else {
            output[j++] = input[i];
        }

        i++;
    }

    output[j] = '\0';

    printf("Parsed Output: %s", output);

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ |

```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

saikrishna@DESKTOP-LGFVDTJ:~$ nano skill5.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc skill5.c -o skill5
saikrishna@DESKTOP-LGFVDTJ:~$ ./skill5
Parent Process PID: 889
Created Child PID: 890
Child Process PID: 890
File1.txt      Pacticalweek4.c  practicalweek3    skill    skill3.c  task1    task3.c
File2.txt      Practical        practicalweek3.c  skill1.c skill4    task1.c  task4
Hello.c        Practical.c      practicalweek4    skill2   skill4.c  task2    task4.c
DSSP          hello           praticalweek4.c  skill2.c skill5    task2.c
Pacticalweek4 hello.c          sai.c            skill3   skill5.c  task3
Child exited with status: 0
saikrishna@DESKTOP-LGFVDTJ:~$ cat skill5.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    char *args[] = {"ls", NULL};

    printf("Parent Process PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process PID: %d\n", getpid());

        execvp(args[0], args);

        printf("Execution failed\n");
        exit(1);
    }

    else {
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
    }

    return 0;
}

```